

UNIVERSITY OF SOUTH FLORIDA
College of Engineering
Department of Electrical Engineering

COURSE PROJECT REPORT:

Project - Matrix Calculator in C: Addition/Subtraction, Multiplication, Transpose, and Simultaneous Linear Equations

Course: EEL3163C (Computer Tool Lab)

Instructor: Chung Seop Jeong, Ph.D.

Submitted By:

Name: Mayaz Ahmed Bin Rouf

StudentID: U35999688

Mail: mayazahmedbin@usf.edu

Semester: Fall 2025

Date of Submission: 06 December 2025

1. Introduction:

This project implements a menu-driven **matrix calculator in C**.

The program allows the user to:

1. Perform **matrix addition or subtraction**.
2. Perform **matrix multiplication**.
3. Compute the **transpose** of a matrix.
4. **Solve a system of simultaneous linear equations** using Gaussian elimination.

The goal of the assignment is to practice C programming with **2-D arrays, loops, conditional statements, user input validation, and functions**, while also reinforcing basic linear algebra operations. The program is fully interactive and checks dimensional compatibility before performing each operation, so that the user receives clear feedback when matrices cannot be combined.

2. Programming Base Code:

```
#include <stdio.h>
```

```
#include <ctype.h>
```

```
#define MAX 10 /* for linear-equation solver */
```

```
/* ----- Helper functions ----- */
```

```
void printMatrix(int r, int c, double A[r][c])
```

```
{
```

```
    int i, j;
```

```
    for (i = 0; i < r; i++)
```

```

{
    printf("| ");
    for (j = 0; j < c; j++)
    {
        printf("%6.2f ", A[i][j]);
    }
    printf("\n");
}
printf("\n");
}

```

/* ----- Choice (a): Matrix Addition / Subtraction ----- */

```

void addOrSubtractMatrices(void)

```

```

{
    int rA, cA, rB, cB;
    int i, j;

    while (1)
    {
        printf("What is the Row number of your Matrix A?=>");
        scanf("%d", &rA);
        printf("What is the Column number of your Matrix A?=>");
    }
}

```

```
scanf("%d", &cA);
```

```
printf("What is the Row number of your Matrix B?=>");
```

```
scanf("%d", &rB);
```

```
printf("What is the Column number of your Matrix B?=>");
```

```
scanf("%d", &cB);
```

```
if (rA == rB && cA == cB)
```

```
    break;
```

```
printf("Sorry. But your matrices are having dimensional mismatch problem!\n");
```

```
printf("Please, check and retype!\n\n");
```

```
}
```

```
double A[rA][cA], B[rB][cB], C[rA][cA];
```

```
/* read elements of A */
```

```
for (i = 0; i < rA; i++)
```

```
{
```

```
    for (j = 0; j < cA; j++)
```

```
    {
```

```
        printf("Please, type in %d row and %d column element of your Matrix A=>",
```

```
            i + 1, j + 1);
```

```
        scanf("%lf", &A[i][j]);
```

```
}  
}
```

```
/* read elements of B */
```

```
for (i = 0; i < rB; i++)
```

```
{
```

```
    for (j = 0; j < cB; j++)
```

```
    {
```

```
        printf("Please, type in %d row and %d column element of your Matrix B=>",
```

```
               i + 1, j + 1);
```

```
        scanf("%lf", &B[i][j]);
```

```
    }
```

```
}
```

```
/* display A and B exactly like the example */
```

```
printf("Here is Matrix A!\n\n");
```

```
printMatrix(rA, cA, A);
```

```
printf("Here is Matrix B!\n\n");
```

```
printMatrix(rB, cB, B);
```

```
/* ask whether to ADD or SUBTRACT B from A */
```

```

char ch;

printf("DO YOU WANT TO ADD OR SUBTRACT MATRIX B FROM MATRIX A?=>");

scanf(" %c", &ch);

ch = toupper((unsigned char)ch);


for (i = 0; i < rA; i++)
{
    for (j = 0; j < cA; j++)
    {
        if (ch == 'A')    /* ADD */

            C[i][j] = A[i][j] + B[i][j];

        else              /* anything else = SUBTRACT */

            C[i][j] = A[i][j] - B[i][j];
    }
}


printf("Here is The Result!\n\n");

printMatrix(rA, cA, C);

}


/* ----- Choice (b): Matrix Multiplication ----- */


void multiplyMatrices(void)
{

```

```
int rA, cA, rB, cB;
```

```
int i, j, k;
```

```
while (1)
```

```
{
```

```
    printf("What is the Row number of your Matrix A?=>");
```

```
    scanf("%d", &rA);
```

```
    printf("What is the Column number of your Matrix A?=>");
```

```
    scanf("%d", &cA);
```

```
    printf("What is the Row number of your Matrix B?=>");
```

```
    scanf("%d", &rB);
```

```
    printf("What is the Column number of your Matrix B?=>");
```

```
    scanf("%d", &cB);
```

```
    /* For multiplication: columns of A must equal rows of B */
```

```
    if (cA == rB)
```

```
        break;
```

```
    printf("Sorry. But your matrices are having dimensional mismatch problem!\n");
```

```
    printf("Please, check and retype!\n\n");
```

```
}
```

```
double A[rA][cA], B[rB][cB], C[rA][cB];
```

```
/* Read Matrix A elements */
```

```
for (i = 0; i < rA; i++)
```

```
{
```

```
    for (j = 0; j < cA; j++)
```

```
    {
```

```
        printf("Please, type in %d row and %d column element of your Matrix A=>",
```

```
            i + 1, j + 1);
```

```
        scanf("%lf", &A[i][j]);
```

```
    }
```

```
}
```

```
/* Read Matrix B elements */
```

```
for (i = 0; i < rB; i++)
```

```
{
```

```
    for (j = 0; j < cB; j++)
```

```
    {
```

```
        printf("Please, type in %d row and %d column element of your Matrix B=>",
```

```
            i + 1, j + 1);
```

```
        scanf("%lf", &B[i][j]);
```

```
    }
```

```
}
```



```
printf("Here is Matrix A!\n\n");  
printMatrix(rA, cA, A);
```

```
printf("Here is Matrix B!\n\n");  
printMatrix(rB, cB, B);
```

```
/* Perform multiplication C = A * B */
```

```
for (i = 0; i < rA; i++)  
{  
    for (j = 0; j < cB; j++)  
    {  
        C[i][j] = 0.0;  
        for (k = 0; k < cA; k++)  
        {  
            C[i][j] += A[i][k] * B[k][j];  
        }  
    }  
}
```

```
printf("Here is The Result!\n\n");  
printMatrix(rA, cB, C);  
}
```

```
/* ----- Choice (c): Transpose ----- */
```

```
void transposeMatrix(void)
{
    int r, c, i, j;

    printf("What is the Row number of your Matrix ?=>");
    scanf("%d", &r);
    printf("What is the Column number of your Matrix ?=>");
    scanf("%d", &c);

    double A[r][c], T[c][r];

    for (i = 0; i < r; i++)
    {
        for (j = 0; j < c; j++)
        {
            printf("Please, type in %d row and %d column element of your Matrix =>",
                i + 1, j + 1);
            scanf("%lf", &A[i][j]);
        }
    }
}
```

```
/* Print original matrix */
```

```
printf("Here is Matrix !\n\n");
```

```
printMatrix(r, c, A); // uses the same printMatrix() we wrote before
```

```
/* Compute transpose */
```

```
for (i = 0; i < r; i++)
```

```
    for (j = 0; j < c; j++)
```

```
        T[j][i] = A[i][j];
```

```
printf("Here is The TRANSPOSE OF Your Matrix!\n\n");
```

```
printMatrix(c, r, T);
```

```
}
```

```
/* ----- Choice (d): Solve Linear Equations (Gaussian elimination) ----- */
```

```
void solveLinearEquations(void)
```

```
{
```

```
    int rA, cA, rB, cB;
```

```
    int i, j, k;
```

```
    while (1)
```

```
    {
```

```
printf("What is the Row number of your Matrix A?=>");  
scanf("%d", &rA);  
printf("What is the Column number of your Matrix A?=>");  
scanf("%d", &cA);
```

```
printf("What is the Row number of your Matrix B?=>");  
scanf("%d", &rB);  
printf("What is the Column number of your Matrix B?=>");  
scanf("%d", &cB);
```

```
if (rA == cA && rB == 1 && cB == cA)  
    break;
```

```
printf("Sorry. But your matrices are having dimensional mismatch problem!\n");  
printf("Please, check and retype!\n\n");  
}
```

```
int n = cA; /* number of unknowns */
```

```
/* A is n x n, B is 1 x n, augmented matrix is n x (n+1) */
```

```
double A[n][n], B[1][n], aug[n][n + 1], x[n];
```

```
/* ---- Read Matrix A ---- */
```

```

for (i = 0; i < n; i++)
{
    for (j = 0; j < n; j++)
    {
        printf("Please, type in %d row and %d column element of your Matrix A=>",
            i + 1, j + 1);
        scanf("%lf", &A[i][j]);
    }
}

```

```

/* ---- Read Matrix B (row vector) ---- */
for (j = 0; j < n; j++)
{
    printf("Please, type in %d row element of your Matrix B=>", j + 1);
    scanf("%lf", &B[0][j]);
}

```

```

printf("Here is Matrix A!\n\n");
printMatrix(n, n, A);

```

```

printf("Here is Matrix B!\n\n");
printMatrix(1, n, B);

```

```

/* ---- Build augmented matrix [A | b] ---- */
for (i = 0; i < n; i++)
{
    for (j = 0; j < n; j++)
        aug[i][j] = A[i][j];

    /* Constant term for equation i is the i-th element of B row */
    aug[i][n] = B[0][i];
}

/* ---- Gaussian elimination: forward elimination ---- */
for (i = 0; i < n - 1; i++)
{
    if (aug[i][i] == 0.0)
    {
        printf("Zero pivot encountered. System may not have unique solution.\n");
        return;
    }

    for (j = i + 1; j < n; j++)
    {
        double factor = aug[j][i] / aug[i][i];

        for (k = i; k <= n; k++)
            aug[j][k] -= factor * aug[i][k];
    }
}

```

```

    }
}

/* ---- Back substitution ---- */
for (i = n - 1; i >= 0; i--)
{
    x[i] = aug[i][n];

    for (j = i + 1; j < n; j++)
        x[i] -= aug[i][j] * x[j];

    if (aug[i][i] == 0.0)
    {
        printf("Division by zero. System may not have unique solution.\n");

        return;
    }

    x[i] /= aug[i][i];
}

/* ---- Print answers in the assignment style ---- */
if (n == 3)
{
    printf("answers are x = %6.2f, y = %6.2f, and z = %6.2f\n",
           x[0], x[1], x[2]);
}

```

```

else
{
    printf("answers are:\n");

    for (i = 0; i < n; i++)

        printf("x%d = %6.2f\n", i + 1, x[i]);

}
}

/* ----- main ----- */

int main(void)
{
    char choice;

    int running = 1;

    while (running)
    {
        printf("=====\n");

        printf("Matrix Operation Menu\n");

        printf("a) Matrix Addition / Subtraction\n");

        printf("b) Matrix Multiplication\n");

        printf("c) Matrix Transpose\n");

        printf("d) Solve Simultaneous Linear Equations\n");

        printf("e) Exit\n");
    }
}

```



```
printf("Enter your choice (a-e)=>");  
  
scanf(" %c", &choice);  
  
switch (tolower((unsigned char)choice))  
{  
    case 'a':  
        addOrSubtractMatrices();  
        break;  
    case 'b':  
        multiplyMatrices();  
        break;  
    case 'c':  
        transposeMatrix();  
        break;  
    case 'd':  
        solveLinearEquations();  
        break;  
    case 'e':  
        running = 0;  
        break;  
    default:  
        printf("Invalid choice. Please select a, b, c, d, or e.\n");  
}  
}
```

```
    printf("Program terminated.\n");

    return 0;
}
```

3. Program Design and Structure

The program is written in standard C and organized into a set of functions, each responsible for one major feature. A small helper function, `printMatrix`, is used by all the operations to print matrices in a consistent, readable format.

3.1 `printMatrix(int r, int c, double A[r][c])`

This helper function prints any matrix `A` with `r` rows and `c` columns. Each row is displayed between vertical bars, matching the style shown in the project examples. It is reused across all menu choices, which reduces code duplication and keeps the output formatting consistent.

```
/* ----- Helper functions ----- */

void printMatrix(int r, int c, double A[r][c])
{
    int i, j;
    for (i = 0; i < r; i++)
    {
        printf("| ");
        for (j = 0; j < c; j++)
        {
            printf("%6.2f ", A[i][j]);
        }
        printf("| \n");
    }
    printf("\n");
}
```

3.2 Choice (b): Multiplication:

```
Enter your choice (a=e)->b
What is the Row number of your Matrix A?=>2
What is the Column number of your Matrix A?=>3
What is the Row number of your Matrix B?=>2
What is the Column number of your Matrix B?=>3
Sorry. But your matrices are having dimensional mismatch problem!
Please, check and retype!
```

```
What is the Row number of your Matrix A?=>2
What is the Column number of your Matrix A?=>3
What is the Row number of your Matrix B?=>3
What is the Column number of your Matrix B?=>3
Please, type in 1 row and 1 column element of your Matrix A=>-5
Please, type in 1 row and 2 column element of your Matrix A=>1
Please, type in 1 row and 3 column element of your Matrix A=>-2
Please, type in 2 row and 1 column element of your Matrix A=>3
Please, type in 2 row and 2 column element of your Matrix A=>-3
Please, type in 2 row and 3 column element of your Matrix A=>1
Please, type in 1 row and 1 column element of your Matrix B=>1
Please, type in 1 row and 2 column element of your Matrix B=>0
Please, type in 1 row and 3 column element of your Matrix B=>2
Please, type in 2 row and 1 column element of your Matrix B=>-7
Please, type in 2 row and 2 column element of your Matrix B=>10
Please, type in 2 row and 3 column element of your Matrix B=>-3
Please, type in 3 row and 1 column element of your Matrix B=>1
Please, type in 3 row and 2 column element of your Matrix B=>0
Please, type in 3 row and 3 column element of your Matrix B=>5
```

Here is Matrix A!

	-5.00	1.00	-2.00	
	3.00	-3.00	1.00	

Here is Matrix B!

	1.00	0.00	2.00	
	-7.00	10.00	-3.00	
	1.00	0.00	5.00	

Here is The Result!

	-14.00	10.00	-23.00	
	25.00	-30.00	20.00	

=====

3.3 Choice (a): Matrix Addition / Subtraction

```
Enter your choice (a-e)=>a
What is the Row number of your Matrix A?=>2
What is the Column number of your Matrix A?=>3
What is the Row number of your Matrix B?=>2
What is the Column number of your Matrix B?=>3
Please, type in 1 row and 1 column element of your Matrix A=>-5
Please, type in 1 row and 2 column element of your Matrix A=>-3
Please, type in 1 row and 3 column element of your Matrix A=>-1
Please, type in 2 row and 1 column element of your Matrix A=>2
Please, type in 2 row and 2 column element of your Matrix A=>5
Please, type in 2 row and 3 column element of your Matrix A=>7
Please, type in 1 row and 1 column element of your Matrix B=>3
Please, type in 1 row and 2 column element of your Matrix B=>21
Please, type in 1 row and 3 column element of your Matrix B=>1
Please, type in 2 row and 1 column element of your Matrix B=>7
Please, type in 2 row and 2 column element of your Matrix B=>5
Please, type in 2 row and 3 column element of your Matrix B=>2
```

Here is Matrix A!

-5.00	-3.00	-1.00
2.00	5.00	7.00

Here is Matrix B!

3.00	21.00	1.00
7.00	5.00	2.00

DO YOU WANT TO ADD OR SUBTRACT MATRIX B FROM MATRIX A?=>A

Here is The Result!

-2.00	18.00	0.00
9.00	10.00	9.00

3.4 Choice (c): Matrix Transpose

```
Enter your choice (a-e)=>C
What is the Row number of your Matrix ?=>2
What is the Column number of your Matrix ?=>3
Please, type in 1 row and 1 column element of your Matrix =>1
Please, type in 1 row and 2 column element of your Matrix =>2
Please, type in 1 row and 3 column element of your Matrix =>3
Please, type in 2 row and 1 column element of your Matrix =>4
Please, type in 2 row and 2 column element of your Matrix =>5
Please, type in 2 row and 3 column element of your Matrix =>6
Here is Matrix !
```

```
| 1.00  2.00  3.00 |
| 4.00  5.00  6.00 |
```

Here is The TRANSPOSE OF Your Matrix!

```
| 1.00  4.00 |
| 2.00  5.00 |
| 3.00  6.00 |
```

=====

3.5 Choice (d): Solving Simultaneous Linear Equations

```
Enter your choice (a-e)=>d
What is the Row number of your Matrix A?=>3
What is the Column number of your Matrix A?=>3
What is the Row number of your Matrix B?=>1
What is the Column number of your Matrix B?=>3
Please, type in 1 row and 1 column element of your Matrix A=>1
Please, type in 1 row and 2 column element of your Matrix A=>1
Please, type in 1 row and 3 column element of your Matrix A=>1
Please, type in 2 row and 1 column element of your Matrix A=>2
Please, type in 2 row and 2 column element of your Matrix A=>1
Please, type in 2 row and 3 column element of your Matrix A=>3
Please, type in 3 row and 1 column element of your Matrix A=>1
Please, type in 3 row and 2 column element of your Matrix A=>-3
Please, type in 3 row and 3 column element of your Matrix A=>1
Please, type in 1 row element of your Matrix B=>2
Please, type in 2 row element of your Matrix B=>9
Please, type in 3 row element of your Matrix B=>10
Here is Matrix A!
```

```
| 1.00  1.00  1.00 |
| 2.00  1.00  3.00 |
| 1.00 -3.00  1.00 |
```

Here is Matrix B!

```
| 2.00  9.00 10.00 |
```

answers are x = 1.00, y = -2.00, and z = 3.00

=====

2.6 `main(void)`

The `main` function runs a loop that repeatedly shows the **menu**:

- a) Matrix Addition / Subtraction
- b) Matrix Multiplication
- c) Matrix Transpose
- d) Solve Simultaneous Linear Equations
- e) Exit

The user enters a character (`a–e`), and a `switch` statement calls the corresponding function. The program continues until the user selects **e) Exit**, at which point it prints “Program terminated.” and ends.

4. What I Learned from This Project

While working on this project, I learned several important concepts:

1. **Practical use of 2-D arrays in C**

I practiced declaring and using variable-length 2-D arrays to hold matrices of different sizes, and I saw how row and column indices are managed with nested loops.

2. **Input validation and user-friendly error messages**

Implementing dimension checks for each operation helped me understand how important it is to validate user input. The dimensional mismatch messages make the program more robust and easier to use.

3. **Function design and modular programming**

Dividing the program into smaller functions (for addition/subtraction, multiplication, transpose, and solving equations) made the code easier to understand, debug, and extend.

4. **Gaussian elimination in code**

Translating the mathematical steps of Gaussian elimination into C code was one of the most challenging parts. I had to carefully control indices in nested loops, manage the augmented matrix, and avoid division by zero. After debugging, I gained a deeper

understanding of how numerical methods are implemented in practice.

5. **Formatting output neatly**

Using a single `printMatrix` function and matching the exact text style in the assignment (for example, “Here is Matrix A!” and “Here is The Result!”) taught me how small formatting details can make the output clear and professional.

In conclusion, this project strengthened both my **C programming skills** and my **linear algebra intuition**, especially how abstract operations like matrix multiplication and solving linear systems look when translated into actual code.

Youtube Video Link:

<https://youtu.be/1FpBhcEIYLc>